

Measuring RM efficiency

Fedor Nikitin, Antti Tolvanen

May 17, 2013

FINNAIR

Questions

- ▶ How to understand that revenue management done well?
- ▶ How to measure it?
- ▶ How to follow market movements?
- ▶ How to separate market performance and revenue management performance on top of the market.

Caution

- ▶ It is about optimization not forecasting.
- ▶ Forecast is main input.
- ▶ BUT forecasting and it's issues are not discussed here.
- ▶ Extensions over basic framework are welcome.
- ▶ Everything is done already by others.

Classic static leg-based RM model. Notations.

- ▶ Classes $C_1, C_2, \dots, C_n$ from higher to lower.
- ▶ Prices $f_1 > f_2 > \dots > f_n$.
- ▶ Demand $d_1, d_2, \dots, d_n$

$$d_i \sim \mathcal{N}(\mu_i, \sigma_i), \quad i = 1, \dots, n.$$

- ▶ Bookings $b_1, b_2, \dots, b_n$

$$b_1 + \dots + b_n \leq \text{cap} \quad (\text{no overbooking}),$$

where cap is capacity.

- ▶ $p_1, \dots, p_n$ classes allocations (kinda protections).

Classic static leg-based RM model. Task.

Revenue

$$Rev(\omega, p_1, \dots, p_n) = \sum_{i=1}^n f_i \min\{d_i(\omega), p_i\}$$

depends on strategy/control/policy/plan $\{p_i\}$ and is indeed random variable with some p.d.f.

Revenue optimization

$$E[Rev(\omega, p_1, \dots, p_n)] \rightarrow \max,$$

$$p_1 + \dots + p_n \leq cap,$$

$$p_i \geq 0, \quad \forall i$$

P5 knows how to solve it, you don't.

FINNAIR

PODS measure

Stochastic (Greek) = Probabilistic (English)

Greedy = optimization is done by greedy algorithm.

Rev_{upper} — greedy maximum revenue.

Rev_{lower} — greedy minimum revenue.

$Rev = \sum_{i=1}^n f_i * b_i$ — actual revenue.

$$REM = \frac{Rev_{upper} - Rev}{Rev_{upper} - Rev_{lower}}$$

Interpretation. Deterministic case.

Rev_{lower} — no RM done (accept all).

Rev_{upper} — optimal revenue indeed (greedy is optimal strategy).

REM measures how close (in some scaled sense) to optimal revenue you are.

Rev_{lower} and Rev_{upper} are indeed lower and upper bounds on revenue.

Makes sense.

When demand $<$ capacity it gets undefined.

It is OK in deterministic case, because RM is not needed.

Interpretation. Stochastic case.

Rev_{lower} and Rev_{upper} are not lower and upper bounds anymore.

Actual bounds $[0, f_1 * cap]$.

Optimal revenue is hard to find (Bellman, EMSR).

Same issue with low demand flights (measure is undefined).

BUT RM is needed. Accept all is not optimal strategy anymore.

In high demand flights it measures something, but we do not know what.

Yet another measure

- ▶ It has interpretation.
- ▶ Defined always (low and high demand flights).
- ▶ Accounts for statistical properties of demand (variance and class correlations).
- ▶ Potentially could be generalized on network level (?).
- ▶ Could provide other interesting RM measures (e.g. VaR).
- ▶ Computationally not hard $O(nm)$.
- ▶ It is indeed framework, not measure.

Mixed strategies

Revenue with pure RM strategy

$$Rev_p(\omega) = \sum_{i=1}^n f_i \min\{d_i(\omega), p_i\}.$$

Mixed RM strategy is probability measure (distribution) on the set of pure strategies.

$\{p_i(\omega)\}$ is mixed RM strategy/policy/control/plan.

Revenue with mixed RM strategy

$$Rev_m(\omega) = \sum_{i=1}^n f_i \min\{d_i(\omega), p_i(\omega)\}.$$

$$\max_{p_i} E[Rev_p(\omega)] = \max_{p_i(\omega)} E[Rev_m(\omega)] \quad (\text{fact!})$$

FINNAIR

Market characteristics

Suppose $\{p_i^*(\omega)\}$ is blind mixed strategy, i.e. not utilizing any knowledge of the market.

Blind revenue is $Rev_b(\omega) = Rev_m(\omega, p_1^*(\omega), \dots, p_n^*(\omega))$.

Random variable $Rev_b(\omega)$ characterizes the market,

Market performance

$$MP = E[Rev_b(\omega)].$$

Market volatility

$$MV = \sqrt{Var[Rev_b(\omega)]}.$$

Other statistical properties of $Rev_b(\omega)$ could be interpreted and utilized as well.

20 SLOC

```
rev = lambda fs,ds,ps: reduce(lambda x,y: x+y,\
    map(lambda f,d,p: f*min(d,p),fs,ds,ps))

def sample_prots(cap,NCLS,ps):
    if cap == 0: return
    ind = np.random.randint(0,NCLS)
    num = np.random.randint(0,cap+1,1)
    ps[ind] += num
    return sample_prots(cap-num,NCLS,ps)

def simulate():
    revs = []
    for i in range(NRUNS):
        ds = [max(0,np.random.normal(mu,sd,1))\
            for mu,sd in zip(mus,sds)]
        ps = [0]*NCLS
        sample_prots(CAP,NCLS,ps)
        revs.append(float(rev(fs,ds,ps)))
    return revs
```

